

Transforming Signals

Getting $Ax(\alpha t + \beta)$ from $x(t)$ — shift, reverse, scale, amplify

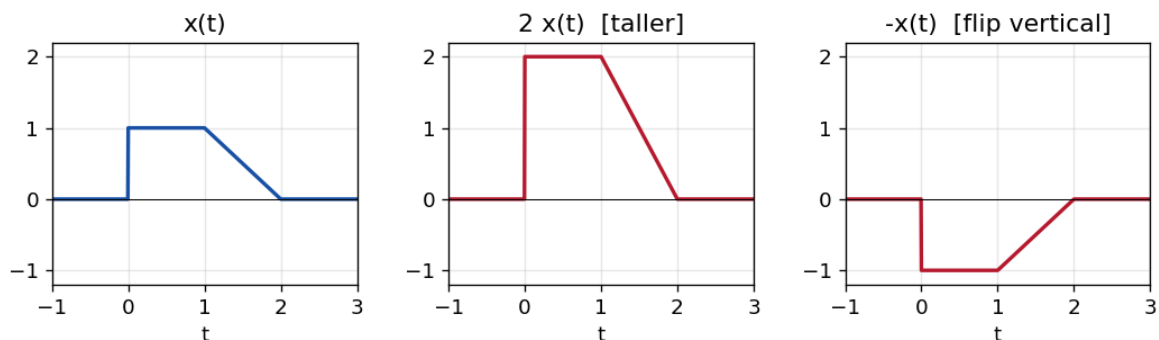
For any α, β : whole, fraction, or negative. Continuous *and* discrete.

1. The Four Building Blocks

Every transformation is a combination of four moves. Two act on the **value** (vertical), two on **time** (horizontal).

Amplitude scaling: $Ax(t)$

Multiply every value by A . Taller if $|A| > 1$, shorter if $|A| < 1$. If $A < 0$ the signal flips *vertically*. The time axis is untouched.



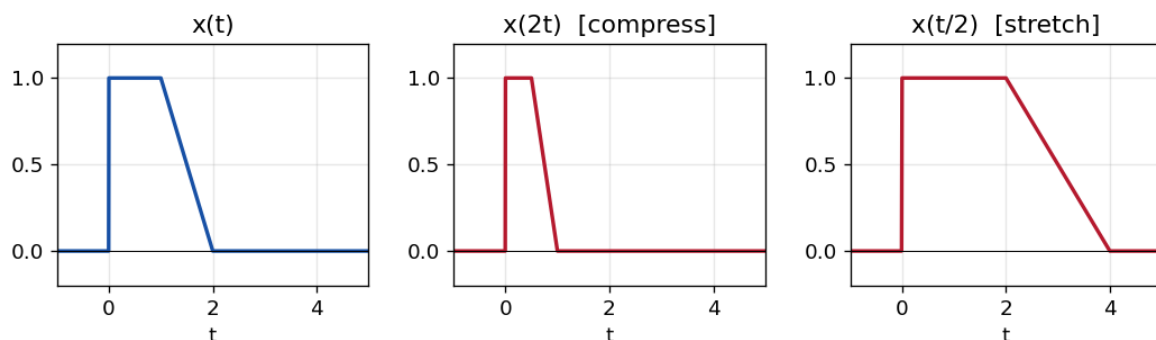
Time shift: $x(t - t_0)$

Moves the whole signal *horizontally*. Watch the sign carefully:

- $x(t - t_0)$ with $t_0 > 0$: shift **right** (delay).
- $x(t + t_0)$ with $t_0 > 0$: shift **left** (advance).

Time scaling: $x(\alpha t)$

Squeezes or stretches along time. $\alpha > 1$ **compresses** (narrower); $0 < \alpha < 1$ **stretches** (wider). A feature at $t = a$ moves to $t = a/\alpha$.



Time reversal: $x(-t)$

Mirror about the vertical axis. A feature at $t = a$ appears at $t = -a$. This is the $\alpha < 0$ case.

2. The Recipe for $x(\alpha t + \beta)$

The slides give a fixed 3-step order. **Follow it exactly and you cannot go wrong**, whatever α, β are (fraction, whole, negative).

To build $x(\alpha t + \beta)$ from $x(t)$:

Step 1 — Shift first. Apply the β shift: $g_1(t) = x(t + \beta)$.

$\beta > 0 \Rightarrow$ shift *left*; $\beta < 0 \Rightarrow$ shift *right*.

Step 2 — Reverse if needed. Only when $\alpha < 0$: $g_2(t) = g_1(-t)$.

Step 3 — Scale by $|\alpha|$. $g_3(t) = g_2(|\alpha|t) = x(\alpha t + \beta)$.

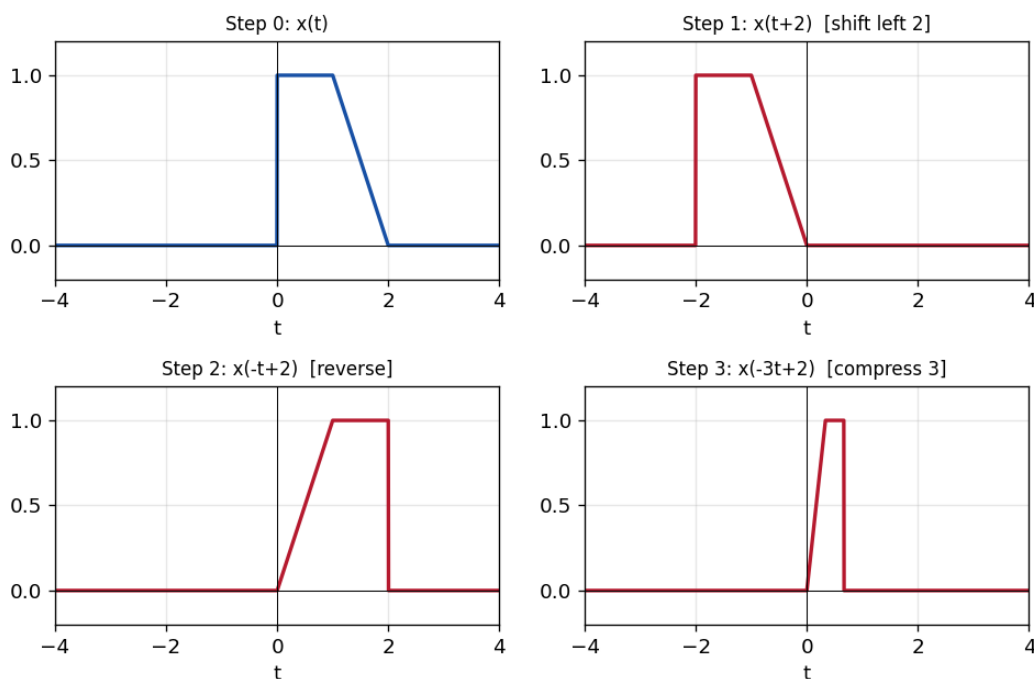
$|\alpha| > 1 \Rightarrow$ compress; $0 < |\alpha| < 1 \Rightarrow$ stretch.

Why shift first? The β inside the recipe is the shift of the *original* variable. If you scale first, the shift amount gets multiplied by α and you must divide to compensate — error-prone. Shifting first sidesteps that entirely. Always write the intermediate signal before the next step.

Worked example: $x(-3t + 2)$ (from the slides)

Here $\alpha = -3$ (negative, whole) and $\beta = 2$.

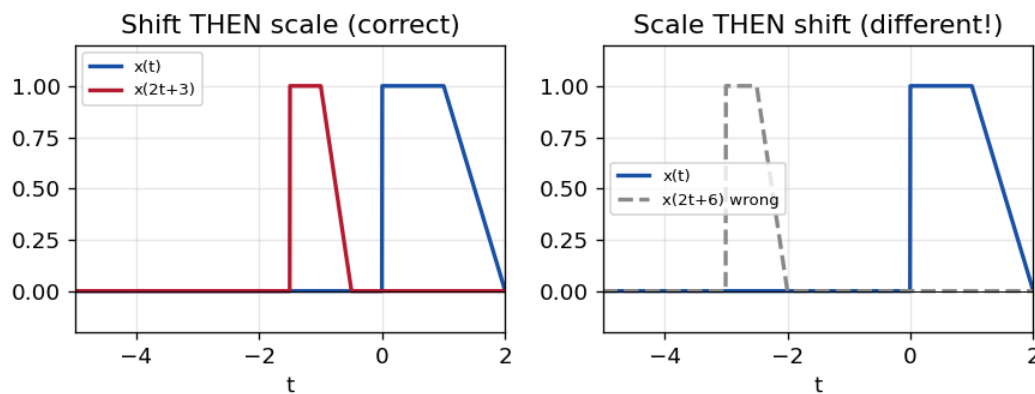
1. Shift left by 2: $g_1(t) = x(t + 2)$.
2. $\alpha < 0$, so reverse: $g_2(t) = x(-t + 2)$.
3. Compress by $|-3| = 3$: $g_3(t) = x(-3t + 2)$. ✓



3. Order Matters — the classic trap

Doing the operations in the *wrong* order gives a *different* signal. Suppose you want $x(2t + 3)$.

- **Right way** (shift then scale): $x(t) \rightarrow x(t + 3) \rightarrow x(2t + 3)$. ✓
- **Wrong way** (scale then shift by 3): $x(t) \rightarrow x(2t) \rightarrow x(2(t + 3)) = x(2t + 6)$. ✗



If a question dictates its own order (e.g. “stretch by 2, then shift left 3, then compress by 4”), do exactly that order and simplify the argument at each step. The 3-step recipe is only for going straight to a target $x(\alpha t + \beta)$.

Following a dictated order (slide Practice 2)

“Stretch by 2, shift left 3, compress by 4, reverse”:

$$x(t) \rightarrow x\left(\frac{t}{2}\right) \rightarrow x\left(\frac{t+3}{2}\right) \rightarrow x\left(\frac{4t+3}{2}\right) = x\left(2t + \frac{3}{2}\right) \rightarrow x\left(-2t + \frac{3}{2}\right)$$

4. Continuous Signals in Code

Your signal is stored as **samples** on a grid `t`. To evaluate x at $\alpha t + \beta$ (which won’t land on grid points), **interpolate**. `np.interp` does linear interpolation in one line.

```
import numpy as np
import matplotlib.pyplot as plt

T_MIN, T_MAX, N = -6.0, 6.0, 2401
t = np.linspace(T_MIN, T_MAX, N)

def x_of_t(t):
    # example: ramp from 1 to 0 over [0,2], else 0
    y = np.zeros_like(t)
    m = (t >= 0) & (t <= 2)
    y[m] = 1.0 - 0.5 * t[m]
    return y

def transform(t, x, A, alpha, beta):
    """y(t) = A * x(alpha*t + beta), for ANY A, alpha, beta."""
    arg = alpha * t + beta # the new argument
    # read x at those (non-grid) points by interpolation;
    # anything outside the original range becomes 0
    return A * np.interp(arg, t, x, left=0.0, right=0.0)

x = x_of_t(t)

# whole / fraction / negative all work the same way:
y1 = transform(t, x, 1.0, 2.0, 0.0) # x(2t)      compress
y2 = transform(t, x, 1.0, 0.5, 0.0) # x(t/2)     stretch
y3 = transform(t, x, 1.0, -1.0, 0.0) # x(-t)      reverse
y4 = transform(t, x, 2.0, 1.0, 3.0)  # 2 x(t+3)   amp + shift
y5 = transform(t, x, -1.5, 0.5, 1.0) # -1.5 x(0.5t+1) everything

plt.figure()
plt.plot(t, x, label="x(t)")
plt.plot(t, y5, label="-1.5 x(0.5t+1)")
plt.xlabel("t"); plt.ylabel("Amplitude")
```

```
plt.grid(True); plt.legend(); plt.show()
```

Why `np.interp` and not array indexing? Shifting/scaling by fractions lands *between* samples. `np.interp(new_arg, t, x)` looks up each new argument on the original curve and linearly interpolates — one clean call that handles fractional α and β automatically. `left=0, right=0` enforces that the signal is 0 outside its known range (“ignore values beyond the range”).

If your course wants nearest-neighbour averaging

Some assignments (the sub-scaling one) specify averaging the two nearest samples instead of linear interpolation. Same idea, explicit:

```
def transform_avg(t, x, A, alpha, beta):
    dt = t[1] - t[0]; t0 = t[0]
    arg = alpha * t + beta
    y = np.zeros_like(t)
    for i, q in enumerate(arg):
        p = (q - t0) / dt          # fractional index
        if p < 0 or p > len(t) - 1:
            continue              # outside range -> 0
        lo, hi = int(np.floor(p)), int(np.ceil(p))
        if np.isclose(p, round(p)): # exact hit (float-safe)
            y[i] = x[int(round(p))]
        else:
            y[i] = 0.5 * (x[lo] + x[hi]) # average two neighbours
    return A * y
```

5. Discrete Signals in Code

For $x[n]$ the index n must stay an integer. So α is restricted:

- **Shift** $x[n - n_0]$ (n_0 integer) and **reversal** $x[-n]$: always fine.
- **Integer scaling** $x[kn]$: downsampling (keeps every k -th sample).
- **Fractional scaling**: not defined without interpolation (upsampling) — usually out of scope; if asked, interpolate like the continuous case.

The safe, general way on a fixed index window is to **look up the source index** for each output index and use 0 if it falls outside.

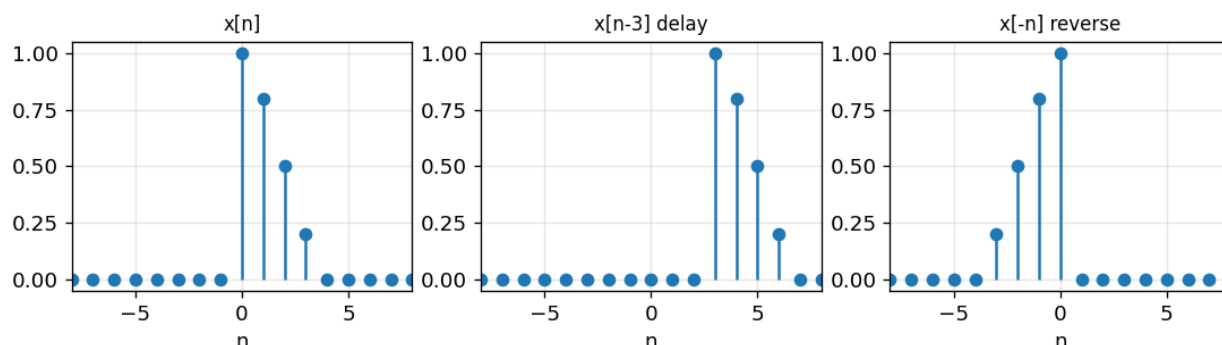
```
n = np.arange(-10, 11)

def x_of_n(n):
    xn = np.zeros_like(n, dtype=float)
    m = (n >= 0) & (n <= 4)
    xn[m] = 4 - n[m]          # 4,3,2,1,0 at n=0..4
    return xn

def transform_discrete(n, x, A, alpha, beta):
    """y[n] = A * x[alpha*n + beta]; alpha integer (+/-), beta integer."""
    y = np.zeros_like(x, dtype=float)
    for i, ni in enumerate(n):
        src = alpha * ni + beta          # required source index
        j = np.where(n == src)[0]       # is it in our window?
        if len(j):
            y[i] = x[j[0]]
    return A * y

xn = x_of_n(n)
y_shift = transform_discrete(n, xn, 1.0, 1, 3) # x[n-3] ? see note
y_delay = transform_discrete(n, xn, 1.0, 1, -3) # x[n-3] delay by 3
y_rev = transform_discrete(n, xn, 1.0, -1, 0) # x[-n] reverse
```

Sign check: a *delay* by 3 is $x[n-3]$, i.e. $\beta = -3$ in $x[\alpha n + \beta]$ with $\alpha = 1$. The value now sitting at index n came from index $n - 3$. Always test on a known signal: `x[n]=[...,4,3,2,1,...]` delayed by 3 should read `[...,0,0,0,4,3,2,1,...]`.



6. Quick Reference & Sanity Checks

Sign & direction table

Operation	Effect
$A > 1$	taller
$A < 0$	flip vertically
$x(t + \beta), \beta > 0$	shift left
$x(t - \beta), \beta > 0$	shift right
$x(\alpha t), \alpha > 1$	compress (narrower)
$x(\alpha t), 0 < \alpha < 1$	stretch (wider)
$x(-t)$	mirror about vertical axis

Feature-tracking check (never fails)

Pick one memorable point of $x(t)$ — say a peak at $t = a$. For $x(\alpha t + \beta)$, that peak moves to the t solving $\alpha t + \beta = a$, i.e. $t = (a - \beta)/\alpha$. If your plotted peak sits there, the transform is right.

Common mistakes

- Shift/scale order swapped $\Rightarrow$ shift multiplied by α . Shift *first*.
- Confusing compress vs stretch: $x(2t)$ is *narrower*, not wider.
- Delay sign flipped: delay is $x(t - t_0)$ / $x[n - n_0]$ with $t_0, n_0 > 0$.
- Forgetting to zero-fill outside the known range (garbage at edges).
- Discrete: using a fractional α without interpolation.
- Float equality on indices $\Rightarrow$ use `np.isclose` / nearest.

Recipe: **shift** $\rightarrow$ **reverse** (if $\alpha < 0$) $\rightarrow$ **scale**. In code: `A * np.interp(alpha*t + beta, t, x)`.